



INDIA.RUNS

Build what next India runs on

Team Name : NullSet

Team Leader Name : Ayush Kumar Singh

Problem Statement : Data & AI Challenge – Intelligent Candidate Discovery

Solution Overview

- What is your proposed solution? What differentiates your approach from traditional candidate matching systems?

A JD-as-rubric ranking engine

The hidden ground truth was built by a careful recruiter (or an LLM) applying THIS job description as a rubric. So we don't guess a similarity score — we faithfully reproduce that recruiter's judgment across all 100,000 candidates, then rank by it.

Four fused signals → one calibrated score

- Dense semantic retrieval (BGE embeddings) + BM25 lexical match capture meaning and exact terms.
- A transparent JD rubric (0–1) encodes the must-haves and disqualifiers explicitly.
- A LightGBM LambdaMART ranker is DISTILLED from an LLM judge that scored a stratified sample with the exact JD — learning the recruiter's weighting rather than us hand-tuning it.
- Multiplicative gates then modify the fit: availability × location × off-target × honeypot.

Why it wins where keyword matchers fail

- Judges by CAREER HISTORY + JOB TITLES, not the skills list — it demotes the keyword-stuffer (a 'Marketing Manager' with every AI buzzword) and promotes the quiet engineer who shipped a recsys at a product company. Honeypot-aware, fully explainable, and it ranks 100k in 40 s on CPU.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD? Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Requirements extracted from the JD

- Production embeddings-based RETRIEVAL (sentence-transformers / BGE / E5) shipped to real users.
- Production VECTOR DB / hybrid search (FAISS, Pinecone, Weaviate, Qdrant, Elasticsearch).
- Ranking-evaluation rigor — NDCG, MRR, MAP, A/B testing, offline→online.
- Strong Python; a scrappy founding-team product-engineer who SHIPS; 5–9 yrs applied ML at PRODUCT companies (not pure IT-services); Pune / Noida.

Signal hierarchy — how we judge fit beyond keywords

- Role & function from titles + trajectory = the DECISIVE signal (encoded as role_target, is_offtarget_current, services_fraction). A wrong-function title vetoes the AI keyword list.
- Evidence of a shipped search / ranking / recsys system — even when never labeled 'RAG' (embedding_retrieval_signal, ever_built_relevant_flag); NLP/IR depth vs. CV/speech-only.
- Behavioral availability is only a MODIFIER. Sentinel values (–1 github, –1 offer-accept, 0 response with 0 applications) mean NO HISTORY → treated NEUTRAL, never penalized.

Ranking Methodology

- How does your system retrieve, score, and rank candidates? What models, algorithms, or heuristics are used? How are multiple candidate signals combined into a final ranking?

Retrieve → Score → Rank

- RETRIEVE: encode every candidate with bge-small-en-v1.5 (384-d); cosine similarity to a JD 'ideal' anchor and 'anti' anchors (marketing / sales / CV-only), plus a BM25 lexical score.
- SCORE: 59 vectorized features → an interpretable rubric (0–1) AND a learned ranker.
- RANK: LightGBM LambdaMART trained on LLM-judge tier labels (0–4) optimizes NDCG directly.

59 engineered features across 8 groups

- Role · experience (fit Gaussian-peaked at 6–8 yrs) · skills (trust-weighted by proficiency×duration, claim-inflation checked vs. assessment scores) · domain signals (vector-DB / embedding / eval-framework / NLP-IR) · company (services-fraction) · location · behavioral (23 signals, sentinels neutralized) · consistency / honeypot.

How signals combine into the final ranking

- $\text{fit} = 0.6 \cdot \text{LambdaMART} + 0.4 \cdot \text{rubric}$, then $\text{final} = \text{fit} \times \text{availability} \times \text{location} \times \text{off-target-gate} \times \text{honeypot-gate}$. Gates can veto (off-target 0.15, honeypot 0.03); modifiers nudge within clamped ranges. Deterministic sort, tie-break by candidate_id ascending.

Explainability & Data Validation

- How are ranking decisions explained? How do you prevent hallucinations or unsupported justifications? How does your solution handle inconsistent, low-quality, or suspicious profiles?

How ranking decisions are explained

- Every candidate gets a fact-grounded reasoning string assembled by template logic over its OWN top contributing factors – real title, years, the systems they built (a phrase quoted verbatim from their best role), domain depth, location, engagement – with tone derived from the score.

Preventing hallucination – by construction, not by prompt

- NO generative model runs at rank time (also a challenge constraint). Each clause is pulled only from a structured field, so a skill can NEVER appear unless it is in the profile. Guarantees: never all-identical, never name-only, never an unsupported claim, concerns stated honestly.

Handling inconsistent / low-quality / suspicious profiles

- A consistency checker flags internal contradictions – role duration > total career, years-of-experience > calendar span, education-vs-experience timeline → honeypot_flag → a 0.03 gate floors them (a >10% honeypot rate in the top-100 means disqualification).
- Sentinels (-1 / 0 = no history) are neutralized, not penalized. Verified: the top-100 contains 0 honeypots, 0 off-target functions, 0 services-only careers, 0 out-of-India candidates.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

PRECOMPUTE – offline, unlimited (network / LLM allowed)

- Stream-parse 100k JSONL → embed all on CPU (BGE-small) → BM25 → a 59-feature store → an LLM judge labels a stratified sample with the JD rubric (free model stack) → train LambdaMART → calibrate weights & gates on a held-out split.
- Artifacts written once: embeddings.npz · jd_query.npz · features.parquet (incl. bm25_score) · labels.parquet · lgbm_ranker.txt.

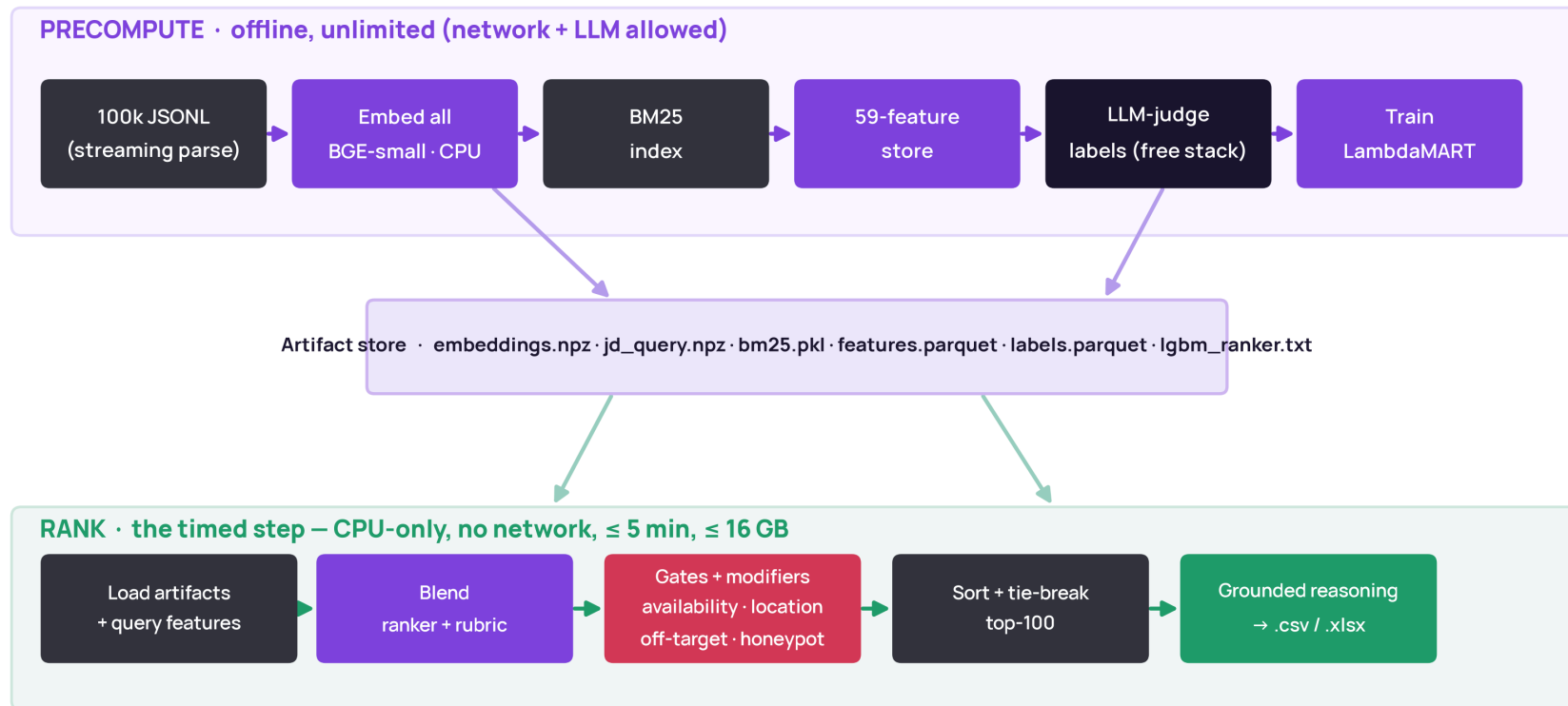
RANK – the timed step (CPU-only, no network, ≤ 5 min, ≤ 16 GB)

- Load artifacts → compute the query-dependent features (dense sim + BM25 = a matmul + lookup) → blend ranker + rubric → apply gates / modifiers → sort & tie-break → top-100 → grounded reasoning → submission.csv + submission.xlsx. Measured: ~40 s, 196 MB on the full 100k.

Robustness

- Fully deterministic (seed-pinned, artifacts committed). Graceful degradation: with no labels/ranker, the rubric-only path still produces a valid, honeypot-clean ranking.

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality? How does your solution meet the challenge's runtime and compute constraints?

What demonstrates ranking quality

- Scored on the official composite ($0.50 \cdot \text{NDCG}@10 + 0.30 \cdot \text{NDCG}@50 + 0.15 \cdot \text{MAP} + 0.05 \cdot \text{P}@10$) against LLM-judge labels: the full hybrid reaches composite 0.65, NDCG@10 0.67, NDCG@50 0.84.
- Ablation tells the story — naive BM25 / keyword match ($\approx$ the sample submission) scores 0.29 AND puts 5 honeypots + 11 out-of-India candidates in its top-100 (a disqualifying ranking). Adding role-gates, semantics and the learned ranker lifts it to 0.68 with a perfectly clean top-100.
- Our consistency layer caught 10 planted timeline-trap candidates that the LLM judge — dazzled by Meta / Apple / Flipkart titles — had rated as top-tier; the math (impossible years-of-experience) does not lie. Feature importance leans on semantic + applied-ML-years + must-haves, as intended.
- Verified top-100 health: 0 honeypots, 0 off-target functions, 0 services-only careers, 0 out-of-India — fully disqualification-safe (a $>10\%$ honeypot rate means DQ).

Meeting the runtime & compute constraints

- Ranking step: ~ 40 s wall-clock on the full 100,000 candidates against the 5-minute budget ($\sim 7.5\times$ headroom); peak memory 196 MB against 16 GB; CPU-only, zero network, fully deterministic.
- Validated by the official `validate_submission.py` ('Submission is valid.'). Embedding precompute ≈ 30 min on a MacBook Pro M3 (one-time, offline).

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?
- Python 3.12 + pydantic v2 — a typed, tolerant schema with a STREAMING JSONL loader (never loads the 465 MB file into memory at once).
- sentence-transformers / BAAI bge-small-en-v1.5 — compact 384-d embeddings that are reliable on CPU at 100k scale (larger models stalled on Apple MPS); dense sim is one signal among 59, so small is the right trade. The harness can A/B a larger model on a GPU.
- rank-bm25 — lexical recall complementing the dense signal.
- LightGBM (LambdaMART) — fast, deterministic, CPU-trained learning-to-rank that optimizes NDCG.
- FREE LLM-judge stack (\$0): Cerebras gpt-oss-120b + Groq (llama-4-scout, llama-3.1-8b, llama-3.3-70b) + Gemini 2.5 — stacked across providers, each its own daily quota, resumable.
- pandas / numpy / pyarrow for vectorized features + a parquet artifact store; Streamlit + Docker for an interactive ranking sandbox.

Submission Assets

- GitHub repository (code + reproducible pipeline): <https://github.com/AyushCoder9/recruitment-signal-fusion>
- Ranked output: team_nullset.xlsx / team_nullset.csv — top-100 (candidate_id · rank · score · reasoning).
- Live sandbox (Hugging Face Spaces): <https://huggingface.co/spaces/ayushxx9/recruitment-signal-fusion> — a Dockerized Streamlit app: upload ≤ 100 candidates → ranked table + per-candidate factor chart + CSV download.
- Reproduce end-to-end: make precompute && make label && make train && make rank && make validate.



INDIA.RUNS

Build what next India runs on

THANK YOU